一、状态图跨复合状态内部节点连接的 mermaid 代码写法

本项目状态图中，复合状态与内部节点的从属关系理应如下：

ROI 绘制：等待操作、框选 ROI、完成 ROI 框选

输入位置编号 1~4：等待放大区域绘制、绘制放大区域(1)、完成放大区域绘制(1)

输入位置编号 5~8：绘制放大区域(2)、完成放大区域绘制(2)

然而，初期编写的 Mermaid 代码所生成的状态图存在一个问题：部分本应显示在复合状态内部的节点，被错误地渲染在了复合状态外部，未能正确体现其从属关系。

```
stateDiagram-v2
    [*] --> 图像加载
    图像加载 --> ROI 绘制和区域放大主循环: 进入绘制界面

    state ROI 绘制和区域放大主循环 {
        direction TB

        state ROI 绘制 {
            direction LR

            [*] --> 等待操作
            等待操作 --> 框选 ROI: 左键按下并拖动
            框选 ROI --> 完成 ROI 框选: 左键释放
            完成 ROI 框选 --> 等待操作: 右键：撤销 ROI
        }

        state 键盘输入 <<choice>>
        完成 ROI 框选 --> 等待键盘输入位置编号
        等待键盘输入位置编号 --> 键盘输入
        键盘输入 --> 输入位置编号 1~4: 键盘输入 1~4
        键盘输入 --> 输入位置编号 5~8: 键盘输入 5~8

        state 输入位置编号 1~4{
            direction LR

            绘制放大区域（位置编号 1~4）: 绘制放大区域
            完成放大区域绘制（位置编号 1~4）: 完成放大区域绘制

            [*] --> 等待放大区域绘制
            等待放大区域绘制 --> 绘制放大区域（位置编号 1~4）: 左键按下并拖动
            绘制放大区域（位置编号 1~4） --> 完成放大区域绘制（位置编号 1~4）: 左键释放
            完成放大区域绘制（位置编号 1~4） --> 等待放大区域绘制: 右键：撤销放大区域
```

```
        完成放大区域绘制（位置编号 1~4） --> [*]
    }

    等待放大区域绘制 --> 等待键盘输入位置编号: 右键

    state 输入位置编号 5~8{
        direction LR

        绘制放大区域（位置编号 5~8）: 绘制放大区域
        完成放大区域绘制（位置编号 5~8）: 完成放大区域绘制

        [*] --> 绘制放大区域（位置编号 5~8）
        绘制放大区域（位置编号 5~8） --> 完成放大区域绘制（位置编号 5~8）
        完成放大区域绘制（位置编号 5~8） --> [*]
    }

    完成放大区域绘制（位置编号 5~8） --> 等待键盘输入位置编号: 右键：撤销放大区域

    state 合并 <<join>>
    输入位置编号 1~4 --> 合并
    输入位置编号 5~8 --> 合并
    合并 --> 等待操作: 继续下一轮 ROI 绘制和区域放大
}

ROI 绘制和区域放大主循环 --> 最终确认: 按 S 键 (完成所有 ROI)
最终确认 --> 批处理所有图片: 用户确认 (Y)
最终确认 --> 程序退出: 用户取消 (N)
ROI 绘制和区域放大主循环 --> 程序退出: 按 Q 键退出
```

该未完善的 mermaid 代码生成的状态图如下：

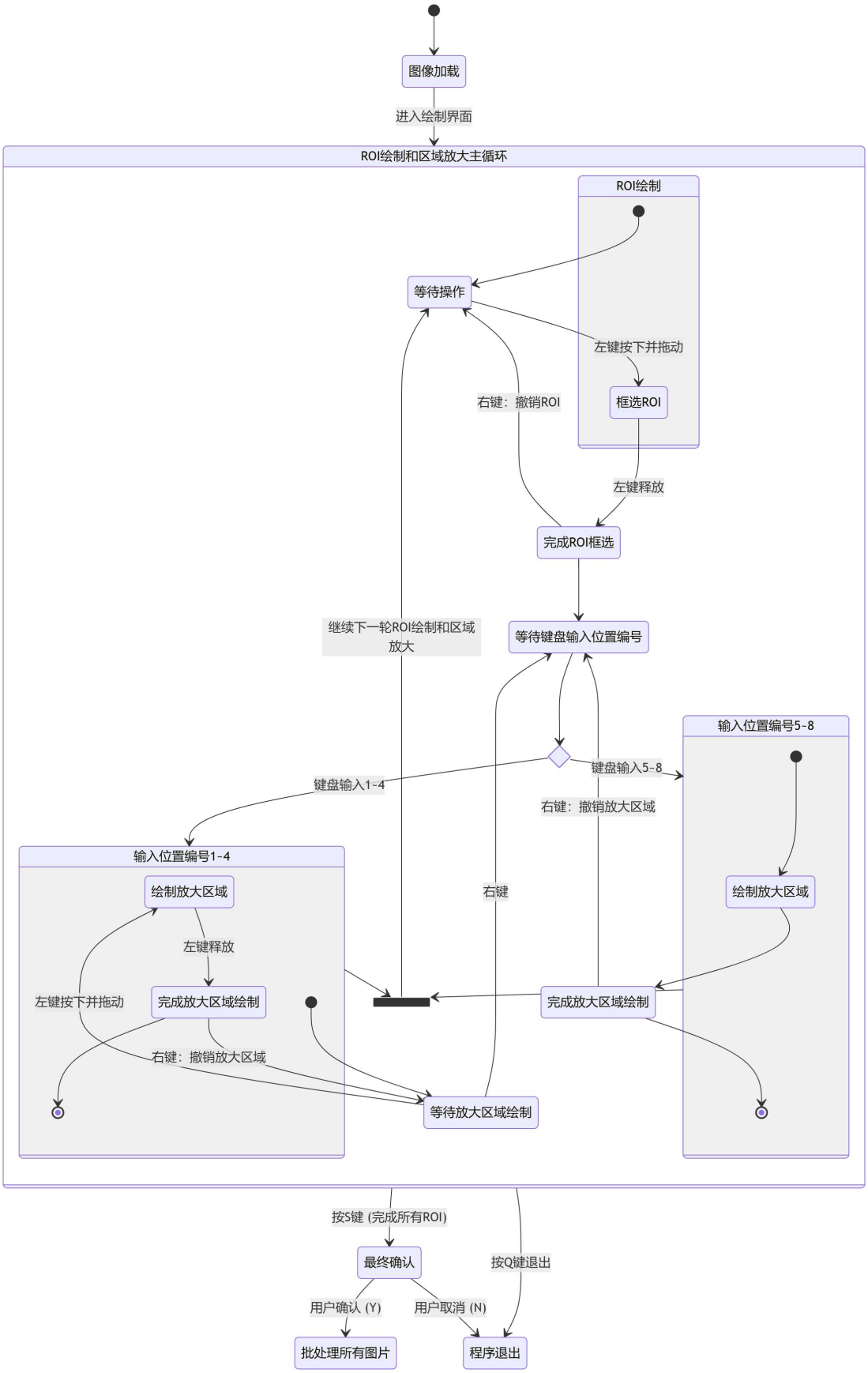


图 1 ROI_zoom2 状态图（不完善版本）

该图与正确显示从属关系的状态图在“ROI 绘制与区域放大”模块中的主要区别如下（红框标出了从属关系不正确的节点）：

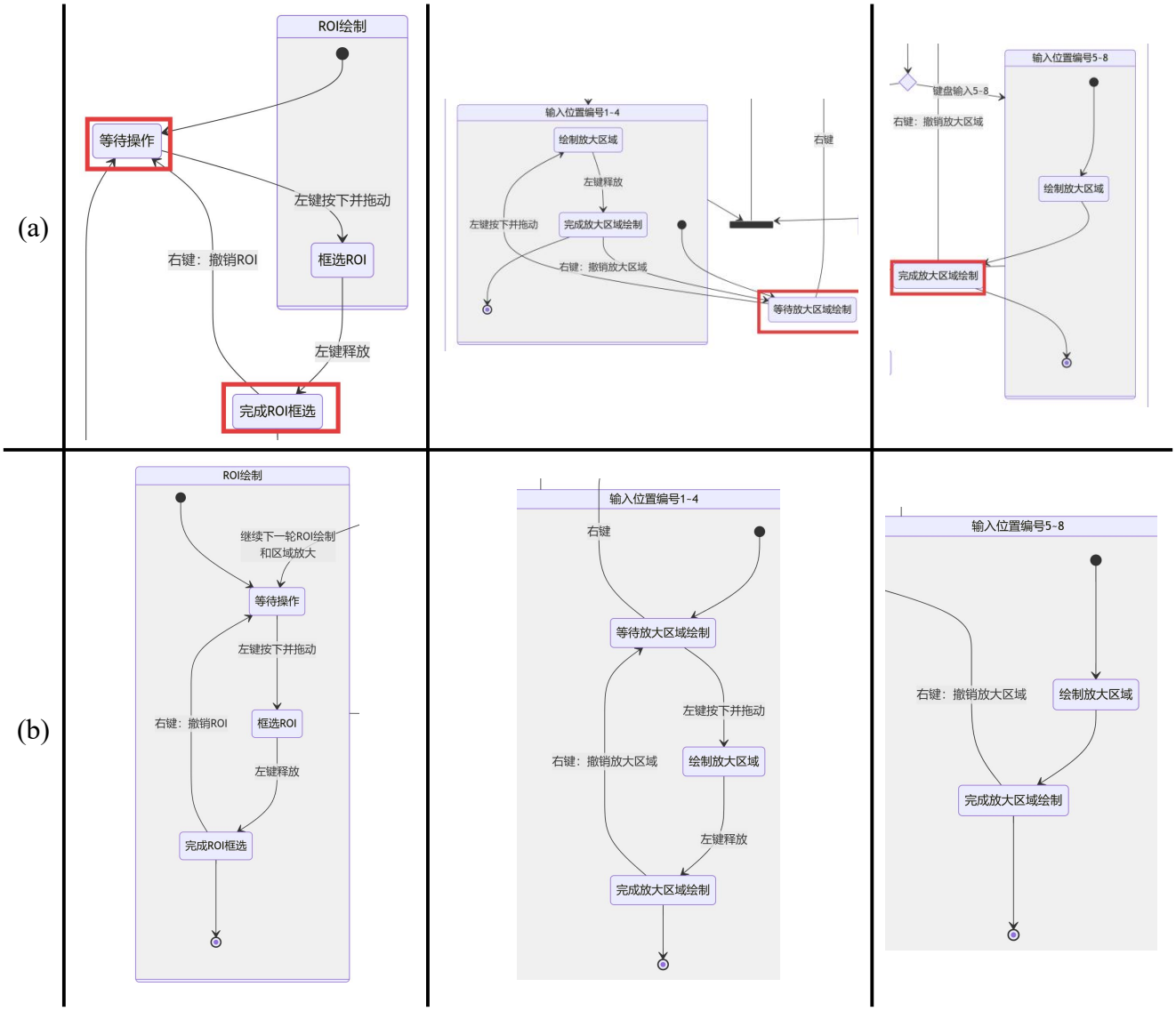


图 2 不完善的状态图与正确显示的状态图的区别

(a) 错误的从属关系；(b)正确的从属关系

因此，我们的目标是：在确保节点被正确绘制在其所属复合状态内部的前提下，实现跨不同复合状态的内部节点之间的连接。针对此问题，本人曾在 DeepSeek、腾讯元宝、CSDN 及 Mermaid 官方文档（如 Mermaid Live Editor）等多个平台进行查询。然而，现有资料主要描述一般节点间的连接，并未提供直接描述此类跨复合状态内部节点连接关系的官方语法。此后，通过自行探索与代码调试，本人最终归纳出了实现该需求的代码写法。

核心发现与解决方案：

Mermaid 渲染器在处理节点从属关系时，后出现的定义会覆盖先前的定义。代码的书写顺序直接决定了节点的最终显示位置。

错误顺序示例：若先定义复合状态“state ROI 绘制 { [*] --> 等待操作 ... }”（此时“等待操作”属于“ROI 绘制”），再编写连接语句“合并 --> 等待操作”，后一句会隐式地将“等待操作”重新定义为**独立节点**，从而覆盖其原有的从属关系，导致其被绘制在复合状态之外。

正确顺序示例：若先编写连接语句“合并 --> 等待操作”，再定义复合状态“state ROI 绘制 { [*] --> 等待操作 ... }”，则后出现的复合状态定义会覆盖“等待操作”作为独立节点的属性，使其被正确收纳于复合状态内部，同时连接关系依然有效。

结论：通过调整代码顺序——**将定义“节点属于某复合状态”的语句，放置在描述“该节点与其他节点连接关系”的语句之后**——即可利用 Mermaid 的覆盖机制，在状态图中同时实现：1) 正确的节点从属关系；2) 正确的节点间连接关系。

基于此方法修正后的完整代码如下：

```
stateDiagram-v2
    direction TB

    [*] --> 图像加载
    图像加载 --> ROI 绘制与区域放大: 进入绘制界面

    state ROI 绘制与区域放大 {
        direction TB

        state 合并 <<join>>
        输入位置编号 1~4 --> 合并
        输入位置编号 5~8 --> 合并
        合并 --> 等待操作: 继续下一轮 ROI 绘制<br>和区域放大

        state ROI 绘制 {
            [*] --> 等待操作
            等待操作 --> 框选 ROI: 左键按下并拖动
            框选 ROI --> 完成 ROI 框选: 左键释放
            完成 ROI 框选 --> 等待操作: 右键：撤销 ROI
            完成 ROI 框选 --> [*]
        }
```

```
    ROI 绘制 --> 等待键盘输入位置编号

    state 键盘输入 <<choice>>
    等待键盘输入位置编号 --> 键盘输入
    键盘输入 --> 输入位置编号 1~4: 输入 1~4
    键盘输入 --> 输入位置编号 5~8: 输入 5~8

    等待放大区域绘制 --> 等待键盘输入位置编号: 右键

    state 输入位置编号 1~4{
        绘制放大区域（位置编号 1~4）: 绘制放大区域
        完成放大区域绘制（位置编号 1~4）: 完成放大区域绘制

        [*] --> 等待放大区域绘制
        等待放大区域绘制 --> 绘制放大区域（位置编号 1~4）: 左键按下并拖动
        绘制放大区域（位置编号 1~4） --> 完成放大区域绘制（位置编号 1~4）: 左键释放
        完成放大区域绘制（位置编号 1~4） --> 等待放大区域绘制: 右键：撤销放大区域
        完成放大区域绘制（位置编号 1~4） --> [*]
    }

    完成放大区域绘制（位置编号 5~8） --> 等待键盘输入位置编号: 右键：撤销放大区域

    state 输入位置编号 5~8{
        绘制放大区域（位置编号 5~8）: 绘制放大区域
        完成放大区域绘制（位置编号 5~8）: 完成放大区域绘制

        [*] --> 绘制放大区域（位置编号 5~8）
        绘制放大区域（位置编号 5~8） --> 完成放大区域绘制（位置编号 5~8）
        完成放大区域绘制（位置编号 5~8） --> [*]
    }

}

ROI 绘制与区域放大 --> 图片批处理与保存

state 图片批处理与保存{
    direction LR

    [*] --> 完成所有 ROI 和放大区域绘制
    完成所有 ROI 和放大区域绘制 --> 预览最终绘制效果: 按 S 键
    state 确认是否批处理所有图片 <<choice>>
    预览最终绘制效果 --> 确认是否批处理所有图片
    确认是否批处理所有图片 --> 批处理所有图片: 用户确认 (Y)
    确认是否批处理所有图片 --> 程序结束: 用户取消 (N)
    [*] --> 程序结束: 按 Q 键退出
```

```
        state 是否保存图片 <<choice>>
        批处理所有图片 --> 是否保存图片
        是否保存图片 --> 保存图片: 确认保存图片 (Y)
        是否保存图片 --> 程序结束: 取消保存图片 (N)
        保存图片 --> 程序结束
        程序结束 --> [*]
    }
```

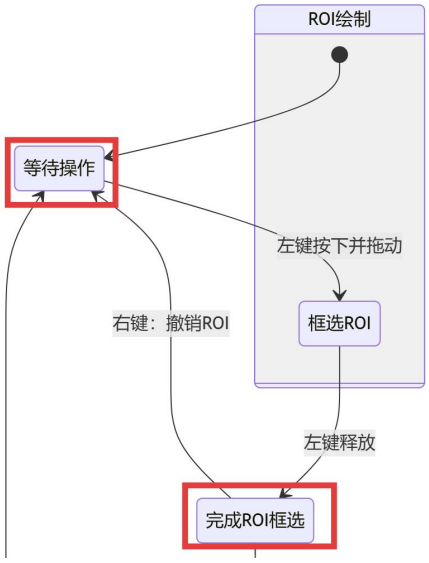
修改前后的关键代码对比如下：

修改前：

```
state ROI 绘制和区域放大主循环 {
        direction TB

        state ROI 绘制 {
            direction LR

            [*] --> 等待操作
            等待操作 --> 框选 ROI: 左键按下并拖动
            框选 ROI --> 完成 ROI 框选: 左键释放
            完成 ROI 框选 --> 等待操作: 右键：撤销 ROI
        }
…
完成 ROI 框选 --> 等待键盘输入位置编号
…
合并 --> 等待操作: 继续下一轮 ROI 绘制和区域放大
```

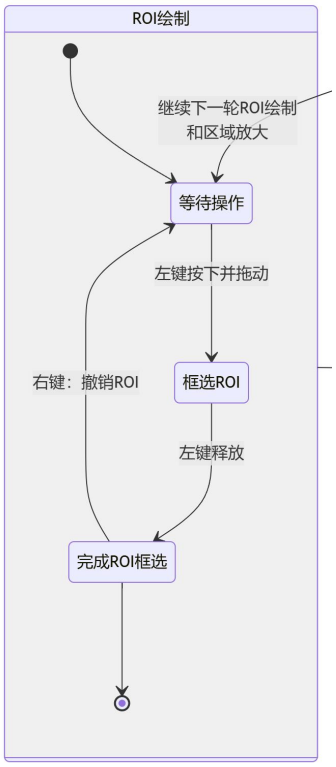


修改后：

```
合并 --> 等待操作: 继续下一轮 ROI 绘制<br>和区域放大

 state ROI 绘制 {
            [*] --> 等待操作
            等待操作 --> 框选 ROI: 左键按下并拖动
            框选 ROI --> 完成 ROI 框选: 左键释放
            完成 ROI 框选 --> 等待操作: 右键：撤销 ROI
            完成 ROI 框选 --> [*]
        }

…
```



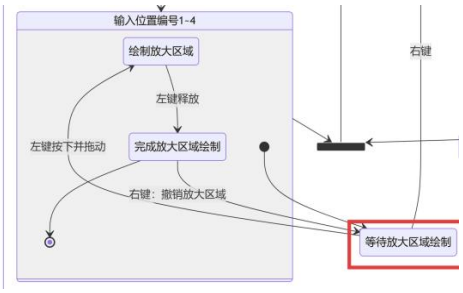
修改前：

state 输入位置编号 1~4{
direction LR

绘制放大区域（位置编号 1~4）：绘制放大区域
完成放大区域绘制（位置编号 1~4）：完成放大区域绘制

[*] --> 等待放大区域绘制
等待放大区域绘制 --> 绘制放大区域（位置编号 1~4）：左键按下并拖动
绘制放大区域（位置编号 1~4） --> 完成放大区域绘制（位置编号 1~4）：左键释放
完成放大区域绘制（位置编号 1~4） --> 等待放大区域绘制: 右键：撤销放大区域
完成放大区域绘制（位置编号 1~4） --> [*]
}

等待放大区域绘制 --> 等待键盘输入位置编号: 右键

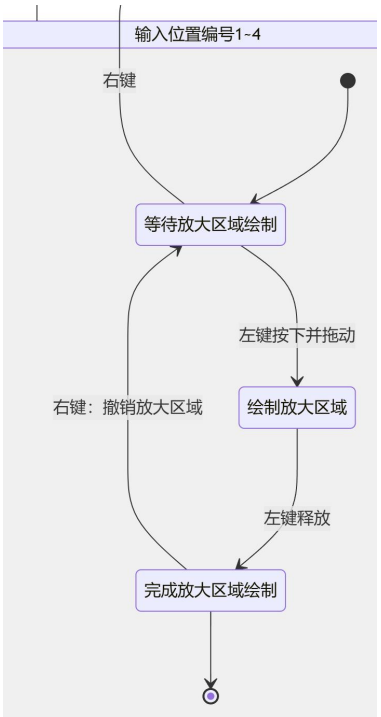


修改后：

等待放大区域绘制 --> 等待键盘输入位置编号: 右键

state 输入位置编号 1~4{
绘制放大区域（位置编号 1~4）：绘制放大区域
完成放大区域绘制（位置编号 1~4）：完成放大区域绘制

[*] --> 等待放大区域绘制
等待放大区域绘制 --> 绘制放大区域（位置编号 1~4）：左键按下并拖动
绘制放大区域（位置编号 1~4） --> 完成放大区域绘制（位置编号 1~4）：左键释放
完成放大区域绘制（位置编号 1~4） --> 等待放大区域绘制: 右键：撤销放大区域
完成放大区域绘制（位置编号 1~4） --> [*]
}



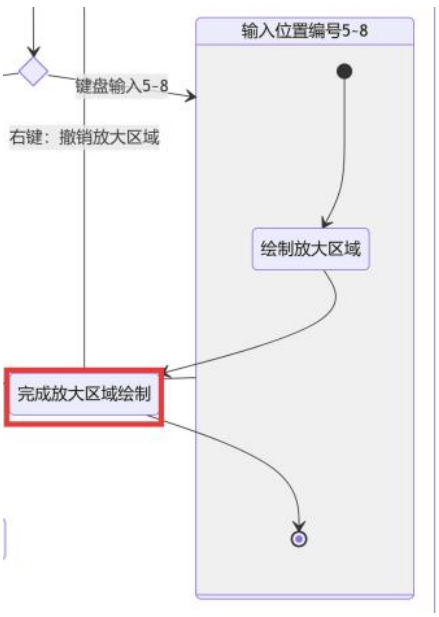
修改前：

state 输入位置编号 5~8{
direction LR

绘制放大区域（位置编号 5~8）：绘制放大区域
完成放大区域绘制（位置编号 5~8）：完成放大区域绘制

[*] --> 绘制放大区域（位置编号 5~8）
绘制放大区域（位置编号 5~8） --> 完成放大区域绘制（位置编号 5~8）
完成放大区域绘制（位置编号 5~8） --> [*]
}

完成放大区域绘制（位置编号 5~8） --> 等待键盘输入位置编号:
右键：撤销放大区域

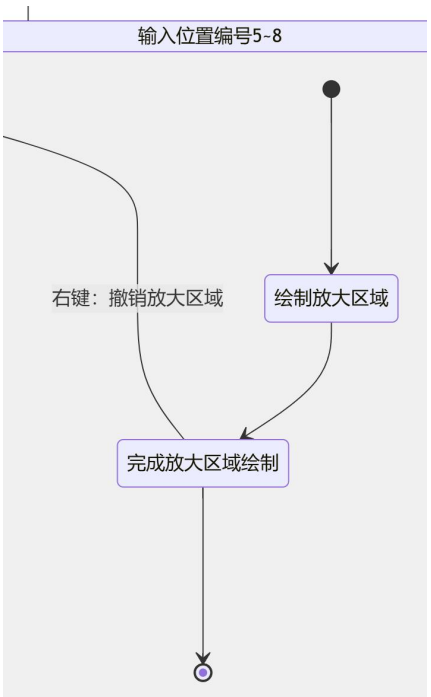


修改后：

完成放大区域绘制（位置编号 5~8） --> 等待键盘输入位置编号:
右键：撤销放大区域

state 输入位置编号 5~8{
绘制放大区域（位置编号 5~8）：绘制放大区域
完成放大区域绘制（位置编号 5~8）：完成放大区域绘制

[*] --> 绘制放大区域（位置编号 5~8）
绘制放大区域（位置编号 5~8） --> 完成放大区域绘制（位置编号 5~8）
完成放大区域绘制（位置编号 5~8） --> [*]
}



修正后的代码所生成的状态图如下，可见所有节点均已正确显示在其所属的复合状态框内：

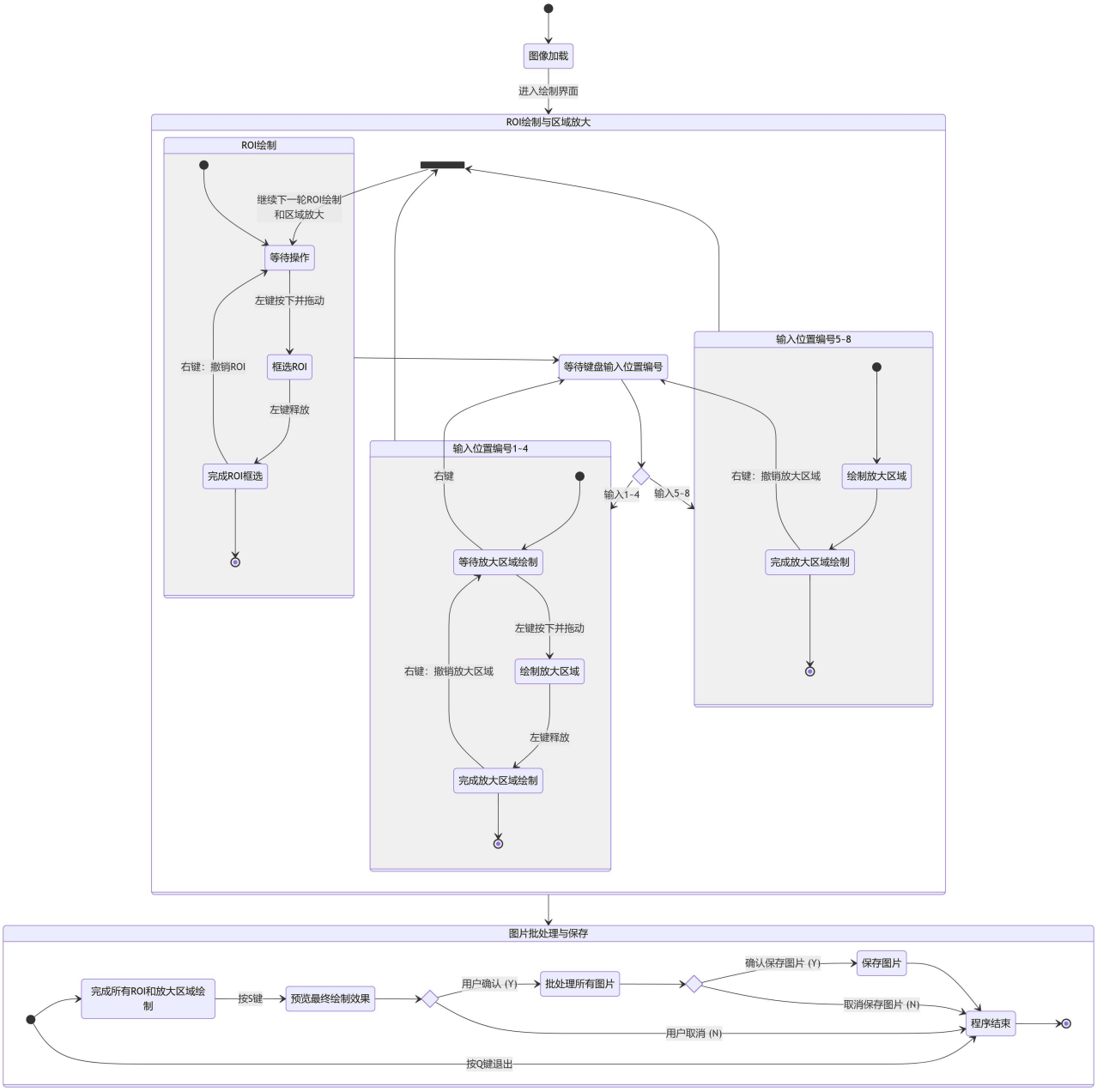


图 3 ROI_zoom2 状态图

二、其他注意事项

1. 状态图中分支语句（<<choice>>）的代码顺序

在使用 <<choice>>关键字定义选择节点时，代码顺序同样会影响渲染结果。

正确顺序：先声明节点类型，再建立连接。此时节点会正确渲染为**菱形**。

例如：以下语句会使得“确认是否批处理所有图片”这种分支节点显示菱形：

```
state 确认是否批处理所有图片 <<choice>>
        预览最终绘制效果 --> 确认是否批处理所有图片
```

错误顺序：先建立连接，再声明节点类型。此时 Mermaid 会将其视为普通节点（含文本），渲染为**圆角矩形**。

因此，在使用 <<choice>>语句时，必须注意将其声明语句置于任何涉及该节点的连接语句之前，以确保图形渲染符合预期。